

Numerical Simulation Methods And Their Applications

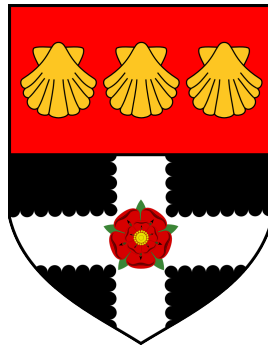
Monte Carlo Methods and Applications in Particle-based Systems

Ridhwaan Amin

32005096

Department of Mathematics and Statistics

University of Reading



May 12, 2026

Plain language summary

This project concerns working with a computer simulation of a particle-based system, particularly using numerical techniques known as Monte Carlo simulations.

It starts with an introduction to the history of these methods, touching upon the Manhattan Project, then focusing on pseudo-random number generators, sampling methodologies, and their applications in statistical mechanics. The project also explores the Metropolis method and its application to a Lennard-Jones liquid. Finally, it concludes with findings of a simulation created from individually developed C++ code.

These findings consist of elements such as total energy within the system, and values derived from numerical analysis of the particles, such as (but not limited to) the radial distribution and diffusion coefficients.

Declaration

I certify that this is my own work and that use of material from other sources has been properly and fully acknowledged in the text. I have read the University's definition of plagiarism and the department's advice on good academic practice. I understand that the consequence of committing plagiarism, if proven and in the absence of mitigating circumstances may include failure in the Year or Part or removal from the membership of the University. I also certify that neither this piece of work, nor any part of it, has been submitted in connection with another assessment.

In submitting this work I can confirm that I **[have/have not]** used Generative AI Tools to prepare and complete my assignment.

to prepare and complete work, please one select the following statements:

~~/a./I acknowledge the use of **[X]** to generate materials to support my research and understanding on the topic when drafting this assignment, but none of the outputs are included in the final version./~~

~~/b./I acknowledge the use of **[X]** to generate materials that were included within my final assessment in modified form./I have taken responsibility to ensure that where generated materials are included, these have been referenced appropriately, in accordance with the requirements of the assignment brief./~~

Contents

List of Tables	v
List of Figures	v
1 Development Of Computer Simulation Methods	1
1.1 The Manhattan Project	1
1.1.1 Background Context	1
1.1.2 Stanisław Ulam	1
1.1.3 Subsequent Developments	1
2 Monte Carlo Methods	2
2.1 Pseudo-Random Number Generators	2
2.1.1 The Linear Congruential Generator (LCG)	2
2.1.2 Linear Congruential Generator Example	3
2.1.3 The Mersenne Twister	3
2.2 Sampling Methodology	4
2.2.1 Simple Sampling	4
2.2.2 Importance	4
2.3 Statistical Mechanics	5
2.4 The Metropolis Method	5
3 The Lennard-Jones Potential	6
3.1 A Description	6
3.1.1 Displaying The Potential Visually	6
3.1.2 Applications in Modelling Simple Liquids	7
3.2 Periodic Boundary Conditions	8
4 Monte Carlo Simulation: Code And Development	9
4.1 Steps Toward Implementation	9
4.1.1 Drawing Together The Theory	9
4.1.2 Initial Configuration	9
4.1.3 Trajectory Generation Loops	9
4.1.4 Post-Simulation Data Analysis	9
4.2 Code Origin and Inspiration	9
5 Application of Code to a Lennard-Jones liquid	10
5.1 Total Energy in Plots	10

5.2	Numerical Analysis	10
5.2.1	Radial Distribution Function	10
5.2.2	Mean-Squared Displacements	10
5.2.3	Diffusion Coefficients	10
6	Conclusions, Findings, Prospectives	11
7	Bibliography	12
A	Approximation of Pi in Python, Using Monte Carlo Techniques	14
B	Approximation of Pi in Python, Code Outputs	16
C	Lennard-Jones Potential, Simple Plot in Python	17
D	Jensen's Inequality	18
E	Monte Carlo C++ Code	19

List of Tables

1	Linear Congruential Generator Example	3
---	--	---

List of Figures

1	Lennard-Jones Potential Plot	6
2	Lennard-Jones Potential Sketch	7
3	Periodic Boundary Conditions	8
4	Total Energy of Simulation	10

1 Development Of Computer Simulation Methods

The first concepts of Monte Carlo simulations can be found in the 18th century, contained within a report found in the 1733 edition of 'Histoire de L'Académie Royale des Sciences'. This report - created by Georges Louis Leclerc, Comte de Buffon - makes mention of an approximation of π by throwing needles or rods atop a floor of boards, measuring the fractions that fell on a single board. Of course, this can be recreated in the modern day by a simulation of approximating π using many points (see Appendices A and B). When initially proposed in this report, the field of Monte Carlo methods had not been distinctly named or partitioned (Landau & Binder 2021).

1.1 The Manhattan Project

1.1.1 Background Context

Before understanding Monte Carlo Methods we are bound to mention **The Manhattan Project**. The top-secret research and development program was the first to produce Nuclear Weapons, and was a key playing card in the context of the greater geopolitical climate: World War II.

Funded by nations such as the UK, Canada and, majorly, the USA, the Manhattan Project is often cited as a turning point in human history. One of the sites of the project was Los Alamos, New Mexico - tied intimately with the birth of Monte Carlo methods. Disregarding moral arguments concerning atomic weaponry, a large factor in the awe of the project was the pure advancement in scientific knowledge - building upon previous discoveries of fission.

1.1.2 Stanisław Ulam

Stanisław Ulam was prominent in contributing to the founding of Monte Carlo Methods as a designated methodology, during and after his involvement in The Manhattan Project. Famously, Ulam thought up the concept while recovering in a hospital from surgery. He had been playing solitaire to pass the time and wondered whether the probability of success could be estimated statistically. Rather than approaching the problem from a combinatorial approach (which he had already tried and found unsuccessful) he began to think more 'abstractly' - he had realised that he could simply observe successive games and note outcomes. Crucially for Stanisław Ulam, this could be extended to many simulated trials with the recent advent of computers - which would yield increasingly accurate estimates (Eckhardt 1987).

1.1.3 Subsequent Developments

The link between Ulam's proposed ideas and the following adoption of the statistical approach was a man named John Von Neumann. Perhaps most well-known for his invention of the aptly named Von Neumann architecture (in which programs are stored within a computer much like the data these programs use), he provided the link to the ENIAC - the world's first programmable, electronic digital computer. Von Neumann's computations, refined together with colleagues at Los Alamos laboratory, became the first calculations run on the ENIAC after it had been upgraded to a stored-program paradigm. Nowadays, the computer industry has grown exponentially, and though statistical sampling had already been used in the past, Ulam's thoughts paved the way for the Monte Carlo method to dominate complex statistical problems (Summerscales 2023).

2 Monte Carlo Methods

Having mentioned the basic, essential history, what are Monte Carlo methods, really?

To boil the concept down to its fundamentals, they are simulations and experiments that use randomness in order to solve complex problems. These problems can manifest in different ways, from integrals to systems of equations. Although numerical solutions and standard approximations may yield more exact or precise answers, Monte Carlo methods offer a viable alternative for large dimensionality of integrals and sparse systems of equations, which can often be solved more effectively in terms of cost and time.

2.1 Pseudo-Random Number Generators

Random number generation is an important part of Monte Carlo methods. There are multiple different methods to obtaining random numbers, but the algorithms we are focusing on within this report are called **Pseudo-Random Number Generators** (PRNGs). While they do not generate 'actual' random numbers, what they generate is sufficient enough in variability that it can be used in scenarios where random numbers are needed.

The way that they achieve this is through the use of mathematical formulae, varying in levels of complexity. The algorithms can get increasingly elaborate, yet the simplest can consist of only a few lines of code utilising basic mathematical arithmetic. Sequences are generated which attempt to approximate random numbers, based on an initial starting *seed*, and these sequences will eventually repeat after their *period*. For satisfactory generators, these periods will be sufficiently long enough that the repetitions are often unreachable by realistic samples (Gentle 2003).

While humans can not predict the next number in the sequences that are generated, this is not perfectly computationally secure. This is because if the initial *seed state* is known, an identical sequence can be reproduced. Therefore, the sequences are deterministic and not as unpredictable as systems with true chaos built into them - such as radioactive atom decay, or, famously, *Cloudflare's* wall of lava lamps (Liebow-Feeser 2017).

2.1.1 The Linear Congruential Generator (LCG)

One such example of a PRNG, albeit possibly the simplest one, is the Linear Congruential Generator given in the following form:

$$x_i \equiv (ax_{i-1} + c) \bmod m$$

As can be seen, the equation consists of a simple linear function followed by a modular reduction. Although this may not produce sufficiently long streams of random numbers, and the numbers may not appear to be very random themselves, the Linear Congruential Generator and its alternative forms create the basis of many more powerful PRNGs. These alternative forms are built by modifying certain elements, for example setting $c = 0$ (yielding something known as the Multiplicative Congruential Generator)

Here,

- a is called the 'multiplier'
- c is called the 'increment'

- m is called the ‘modulus’.

The seed for this generator is the initial value supplied, which is to say x_0 . Due to the modular reduction that occurs, there can only be m possible values that x can take; The maximum period of the LCG is, by definition, m (Gentle 2003).

2.1.2 Linear Congruential Generator Example

Following is an example of a LCG displayed in a ‘truth table’, allowing the process to be broken down into small, simple, and visible steps. Traditionally, values of m are usually a power of 2 as this is easier for the computer to use during calculation. Due to this, I have chosen $m = 2^5 = 32$ which is smaller than a typical usecase. The other values that this LCG will take are $a = 5$, $c = 7$, $X_0 = 19$, which have been arbitrarily selected.

Table 1: Linear Congruential Generator Example

aX_n	$+ c$	$(\text{mod } m)$	$= X_{n+1}$
$(5 * 19) = 95$	$(95 + 7) = 102$	$102 \text{ (mod } 32)$	$= 6$
$(5 * 6) = 30$	$(30 + 7) = 37$	$37 \text{ (mod } 32)$	$= 5$
$(5 * 5) = 25$	$(25 + 7) = 32$	$32 \text{ (mod } 32)$	$= 0$
$(5 * 0) = 0$	$(0 + 7) = 7$	$7 \text{ (mod } 32)$	$= 7$
$(5 * 7) = 35$	$(35 + 7) = 42$	$42 \text{ (mod } 32)$	$= 10$
$(5 * 10) = 50$	$(50 + 7) = 57$	$57 \text{ (mod } 32)$	$= 25$
$(5 * 25) = 125$	$(125 + 7) = 132$	$132 \text{ (mod } 32)$	$= 4$

From the table, we can see that for the Linear Congruential Generator $5X_n + 7 \text{ (mod } 2^5)$, the starting seed of $X_0 = 19$ provides a sequence of: 6, 5, 0, 7, 10, 25, 4...

Following this, the sequence continues until it starts to repeat, having a period of 32 (the value of m , as stated earlier). The full sequence in its entirety is:

19, 6, 5, 0, 7, 10, 25, 4, 27, 14, 13, 8, 15, 18, 1, 12, 3, 22, 21, 16, 23, 26, 9, 20, 11, 30, 29, 24, 31, 2, 17, 28, 19, ...

2.1.3 The Mersenne Twister

The Mersenne Twister is a more complex, modern PRNG family, developed initially in 1997 (Matsumoto & Nishimura 1998).

It derives its name from a prime number known as a Mersenne Prime, which contributes to its period. As its period is sufficiently large, the algorithm is highly optimised, and the numbers generated have low correlations, the Mersenne Twister is a prominent PRNG that is used across industry. Most notably, it is used in scientific simulations (Monte Carlo methods, as discussed) but also within Machine Learning, statistical sampling and also video game graphics (Samadov 2024).

As, arguably, the most widely used PRNG in the modern day, the Mersenne Twister can be found in many software libraries for standard random generation. These include, but are not limited to, Python, Ruby, and R. One of its multiple variant forms can be used in C++ through `std::mt19937` in the standard library. Due to these occurrences, it is highly likely that within code used in this report (found in the Appendices), the Mersenne Twister is the PRNG that has contributed random elements. This, however, has not been strictly verified.

2.2 Sampling Methodology

By 'Sampling Methodology', we refer to the way in which Monte Carlo methods are used to sample data points (selections to be resampled within subsequent analysis). Depending on the context and scenario, different qualities may be prioritised, and this is reflected in the different ways that sampling can occur (Brownlee 2019).

The 'Law of Large Numbers' plays a significant part in these simulations as that allows the sampling that occurs to form a relevant distribution, despite the initial data being random.

2.2.1 Simple Sampling

Simple sampling, also called a 'crude monte carlo', is the most direct and rudimentary method of using monte carlo sampling techniques. This entails the collection of completely random observations of a random variable. The aforementioned reliance on the law of large numbers leads to a large enough sample size so that an estimate of the expected value ($E[X]$) can be found (Simple Sampling 2010).

To see the way that simple sampling works, we can base an example upon the integral of a collected distribution. If a one-dimensional integral I , where $I = \int_a^b dx f(x)$, were to be re-written as:

$$I = (b - a) \langle f(x) \rangle,$$

this would represent the unweighted average of $f(x)$ over the interval $[a, b]$. As this average is evaluated, larger and larger distributions, tending to infinity, yield a correct value of I (Frenkel & Smit 1996).

An example of this 'crude monte carlo' method can be found in the approximation of Pi mentioned earlier (see Appendices A and B), where uniform and random coordinates are generated within the given square. This results in a calculated average for the value of Pi. This sampling methodology can fall short due to its reliance on randomness; If random values fell unusually within negligible or heavily influencing areas, the final results of a monte carlo sample can be inaccurate - rendering the process inefficient.

2.2.2 Importance

Like Simple sampling, Importance sampling is a method of sampling for data values over an interval, however what sets Importance sampling apart is the addition of weighted 'importance' to certain selected values (hence the name given to the technique). This is achieved through the introduction of a new weight term which we will designate as $p(x)$. This function is commonly known as the '*importance function*'.

Fulfilling a key objective of sampling in the first place, this leads to the variation being reduced to either insignificant levels or, at best, zero in areas where the distribution may not be relevant to draw samples from.

Representing the probability density over a dimension, D , this importance function can be introduced into our earlier estimated integral, making sure $p(x)$ is carefully chosen to minimise variance:

$$\int_D \frac{f(x)}{p(x)} p(x) dx$$

Variation, with respect to the distribution of X with density $p(x)$, is, hence, given by:

$$V\left(\frac{f(X)}{p(X)}\right) = \left[E\left(\frac{f^2(X)}{p^2(X)}\right)\right] - \left[E\left(\frac{f(X)}{p(X)}\right)\right]^2$$

where $\left[E\left(\frac{f(X)}{p(X)}\right)\right]^2$ is actually equal to $\left(\int_D f(x) dx\right)^2$, suggesting that only the first expression in the Variance formula has any significant effect on the importance sample.

By Jensen's inequality (see Appendix D for theorem and proof via (Wasserman 2004)), there is a lower bound on this first term. This looks like:

$$\begin{aligned} E\left(\frac{f^2(X)}{p^2(X)}\right) &\geq \left(E\left(\frac{|f(X)|}{p(X)}\right)\right)^2 \\ &= \left(\int_D |f(x)| dx\right)^2 \end{aligned}$$

This is where it is important to select a correct $p(x)$. This bound we have shown is achieved when

$$p(x) = \frac{|f(x)|}{\int_D |f(x)| dx},$$

but this is futile when we're considering a monte carlo estimate (and do not know the integral necessarily) - therefore we seek to choose p nearly proportional to $|f|$. Ultimately, it is preferable for $\frac{|f(x)|}{p(x)}$ to be nearly constant (Gentle 2003) as this allows variance to, as stated previous, "vanish altogether" (Frenkel & Smit 1996).

2.3 Statistical Mechanics

In order to understand working with particles, and simulations of them, statistical mechanics plays a pivotal part. Within this field, matters of concern include 'macrostates' and 'microstates'.

(Saxena 2016) (Greiner et al. 1995)

2.4 The Metropolis Method

3 The Lennard-Jones Potential

3.1 A Description

The Lennard-Jones potential describes the potential energy of two non-bonded particles based on their distance of separation. The equation accounts for both attractive forces and repulsive forces.

The equation is given as follows

$$V(r) = 4\varepsilon \left[\left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right]$$

where:

- V indicates the intermolecular potential between particles,
- ε is the well depth: the maximum strength of attractive force and minimum energy value
- σ is referred to as the '**van der Waals**' radius, and is a measure of how close two nonbonding particles can get. It measures the distance at which the intermolecular potential between the two particles is zero.
- r indicates distance of separation between both particles, measured from the center of each.

The σ/r^{12} term models strong, short-range repulsion and the σ/r^6 term models long-range van der Waals attraction.

Sometimes the potential can be rearranged and expressed alternatively as

$$V(r) = \frac{A}{r^{12}} - \frac{B}{r^6}$$

in which A, B are equal to $4\varepsilon\sigma^{12}, 4\varepsilon\sigma^6$ respectively (*Lennard-Jones Potential* 2025).

3.1.1 Displaying The Potential Visually

The Lennard-Jones plot can be shown in a graphical manner, representing the various elements of the equation, as described earlier. Setting $\sigma, \varepsilon = 0$, for simplicities sake, generates the following graph, programmed in python (see Appendix C) using, primarily, the matplotlib library (Hunter 2007).

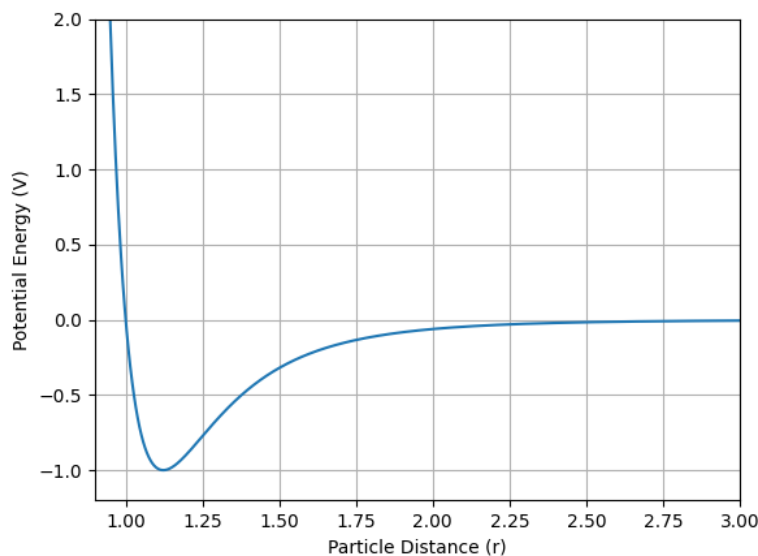


Figure 1: **Lennard-Jones Potential Plot**

In order to facilitate the showcasing of more details on certain parts of the graph, there is also a physical sketch included below. This highlights where certain variables come into play in different parts of the plot, such as σ and ϵ , alongside the repulsion and attraction aspects.

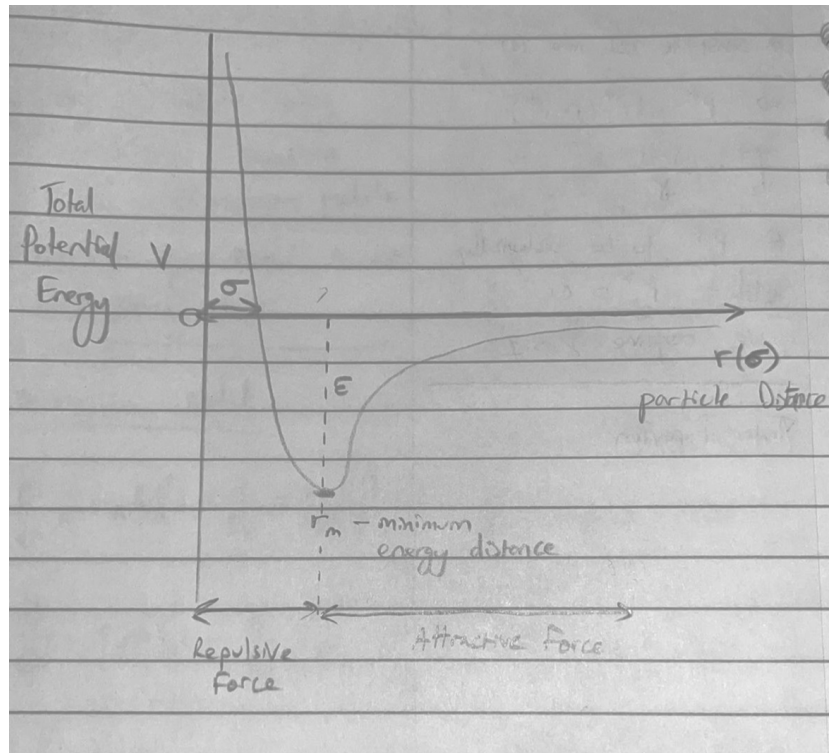


Figure 2: Lennard-Jones Potential Sketch

3.1.2 Applications in Modelling Simple Liquids

When modelling simple liquid simulations, the Lennard-Jones potential is relevant because

3.2 Periodic Boundary Conditions

Boundary conditions help to define how a simulation may handle particles within a system. Periodic boundary conditions, in particular, aid in abstracting a small simulation of particles so that it seems like they are interacting within an infinite system. This is done by letting particles cross over the boundaries of the simulation and into identical neighbouring 'cells', i.e cells are tiled infinitely and particles leaving the cell re-enter through the opposite face which can be seen in **Figure 3** (Katiyar & Jha 2018). This allows a much larger system to be simulated with a smaller computational load (as the simulation is started with fewer particles involved).

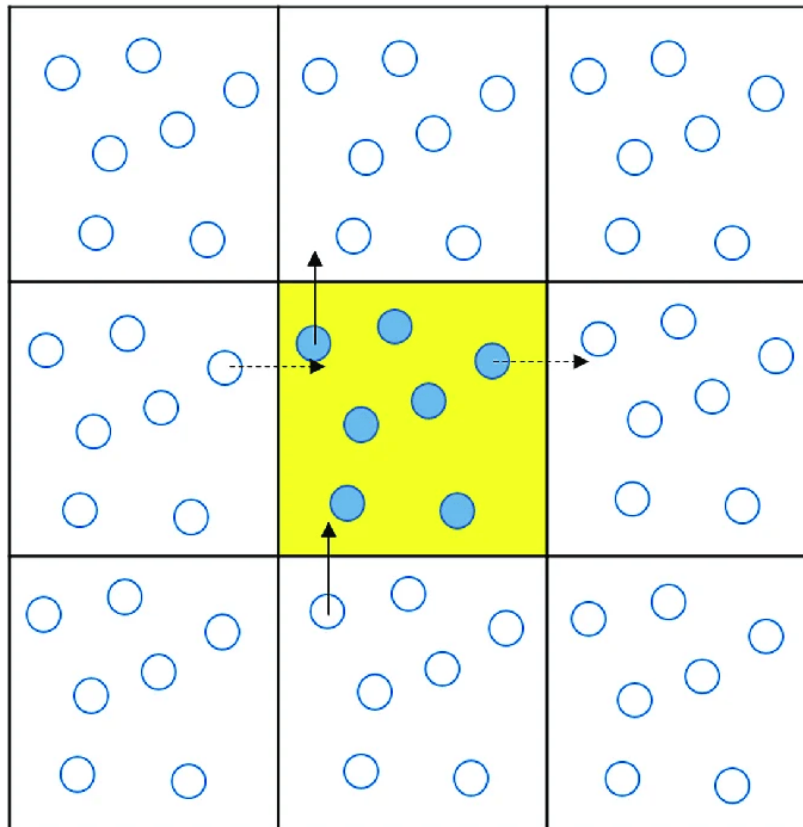


Figure 3: Periodic Boundary Conditions

4 Monte Carlo Simulation: Code And Development

4.1 Steps Toward Implementation

4.1.1 Drawing Together The Theory

In order for correct implementation of the code, all of this theory needs to be drawn together appropriately.

Some of these concepts are built into the software that I am programming with, such as PRNGs, they are very commonly present in random functions that are found in code libraries. As such is the case, I will be using an inbuilt Mersenne Twister within my program code.

The Lennard-Jones potential, previously mentioned, will be used within the code in order for calculations to be taken on particle distances and repulsions. This will be hard-coded in.

4.1.2 Initial Configuration

Mersenne Twister seed is 32005096

4.1.3 Trajectory Generation Loops

4.1.4 Post-Simulation Data Analysis

4.2 Code Origin and Inspiration

My code was built on a framework of **Fortran** code from 'Understanding Molecular Simulation' by Daan Frenkel and Berend Smit (Frenkel & Smit 1996). Another Monte Carlo simulation of the Lennard-Jones model (Kelter et al. 2015), packaged within a library known as NetLogo (Wilensky 1999), also provided much guidance and inspiration. These were adapted and rewritten from scratch, by me, into **C++**. My program can be found in the appendix of this report (see Appendix E).

5 Application of Code to a Lennard-Jones liquid

5.1 Total Energy in Plots

A placeholder for a plot showing the total energy of the simulation. The plot area is represented by a small rectangle.

Figure 4: Total Energy of Simulation

5.2 Numerical Analysis

5.2.1 Radial Distribution Function

5.2.2 Mean-Squared Displacements

5.2.3 Diffusion Coefficients

6 Conclusions, Findings, Prospectives

7 Bibliography

- Brownlee, J. (2019), 'A gentle introduction to monte carlo sampling for probability', <https://machinelearningmastery.com/monte-carlo-sampling-for-probability/>.
- Eckhardt, R. (1987), 'Stan Ulam, John von Neumann, and the Monte Carlo Method', *Los Alamos Science* (15), 131–141. Also: Los Alamos National Laboratory Tech. Report LA-UR-88-9068.
URL: <https://permalink.lanl.gov/object/tr?what=info:lanl-repo/lareport/LA-UR-88-9068>
- Frenkel, D. & Smit, B. (1996), *Understanding Molecular Simulation: From Algorithms to Applications*, Academic Press.
- Gentle, J. E. (2003), *Random Number Generation and Monte Carlo Methods*, second edn, Springer.
- Greiner, W., Neise, L. & Stöcker, H. (1995), *Thermodynamics and Statistical Mechanics*, Springer. Translated from the German by Dirk Rischke.
- Hunter, J. D. (2007), 'Matplotlib: A 2d graphics environment', *Computing in Science & Engineering* **9**(3), 90–95.
- Katiyar, R. & Jha, P. (2018), 'Molecular simulations in drug delivery: Opportunities and challenges', *Wiley Interdisciplinary Reviews: Computational Molecular Science* **8**, e1358.
- Kelter, J., Luijten, E. & Wilensky, U. (2015), 'Netlogo monte carlo lennard-jones model', <http://ccl.northwestern.edu/netlogo/models/MonteCarloLennard-Jones>.
- Kopka, H. & Daly, P. W. (1999), *A Guide to LaTeX*, third edn, Pearson Education.
- Landau, D. P. & Binder, K. (2021), *A Guide to Monte Carlo Simulations in Statistical Physics*, fifth edn, Cambridge University Press.
- Lennard-Jones Potential* (2025). [Online; accessed 2026-04-22].
- Liebow-Feaser, J. (2017), 'Randomness 101: Lavarand in production', <https://blog.cloudflare.com/randomness-101-lavarand-in-production/>.
- Matsumoto, M. & Nishimura, T. (1998), 'Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator', **8**(1).
URL: <https://doi.org/10.1145/272991.272995>
- Metcalfe, T. (2023), 'What was the manhattan project?', *Scientific American*.
- Samadov, I. (2024), 'The mersenne twister algorithm. a deep dive into a pseudo-random number generator', *Medium*.
- Saxena, A. K. (2016), *An Introduction to Thermodynamics and Statistical Mechanics*, second edn, Alpha Science International, Limited, 2016.
- Sharma, V. (2023), 'Estimating pi using monte carlo methods', <https://medium.com/the-modern-scientist/estimating-pi-using-monte-carlo-methods-dbf26c888d6>.
- Simple Sampling* (2010), Lecture Slides for EG2080 Monte Carlo Methods in Engineering.
URL: <https://www.kth.se/social/upload/50bf16b7f276547633d8a01b/Theme%203%20-%20Simple%20Sampling.pdf>
- Summerscales, O. (2023), 'Hitting the jackpot: The birth of the monte carlo method', <https://www.lanl.gov/media/publications/actinide-research-quarterly/1123-hitting-the-jackpot-the-birth-of-the-monte-carlo-method>.

Wang, C. (2018), Applications of Monte Carlo Methods in Studying Polymer Dynamics, PhD thesis, University of Reading.

Wasserman, L. A. (2004), *All of Statistics: A Concise Course in Statistical Inference. Springer Texts in Statistics*, Springer.

Wilensky, U. (1999), 'Netlogo', <http://ccl.northwestern.edu/netlogo/>.

A Approximation of Pi in Python, Using Monte Carlo Techniques

ApproximationOfPi/src/python/SimpleImplementation.py

```
#!/ python3
"""
Simple implementation of approximation of pi.

Steps:
Generate random x, y inside centre of square of side 2
Repeat steps for a large number of points
Calculate ratio of points
Multiply by 4 (pi / 4)
"""

import numpy as np

def estimatePi(samples):
    insideCircle = 0
    for i in range(samples):
        x = np.random.uniform(-1, 1)
        y = np.random.uniform(-1, 1)

        if (x**2 + y**2) <= 1:
            insideCircle += 1
    ratio = insideCircle / samples
    return (ratio * 4)

if __name__ == "__main__":
    print(f"Estimate with 1 Sample:\n{estimatePi(1)}\n")
    print(f"Estimate with 10 Sample:\n{estimatePi(10)}\n")
    print(f"Estimate with 50 Sample:\n{estimatePi(50)}\n")
    print(f"Estimate with 100 Sample:\n{estimatePi(100)}\n")
    print(f"Estimate with 1000 Sample:\n{estimatePi(1_000)}\n")
    print(f"Estimate with 10,000 Sample:\n{estimatePi(10_000)}\n")
    print(f"Estimate with 1,000,000 Sample:\n{estimatePi(1_000_000)}\n")
```

ApproximationOfPi/src/python/GraphicalRepresentation.py

```
#!/ python3
"""
Graphical representation of approximation of Pi using Monte Carlo methods.
"""

import matplotlib.pyplot as plt
import numpy as np

from SimpleImplementation import estimatePi

def main():
    samples = int(input("Enter Intended Samples:\n"))

    x = np.random.uniform(-1, 1, samples)
    y = np.random.uniform(-1, 1, samples)

    plt.plot(x, y, ".k", markersize = 1)
```

```
t = np.linspace(0, 2 * np.pi, 400)
xc = np.cos(t)
yc = np.sin(t)
plt.plot(xc, yc, 'r-', linewidth=1)
plt.gca().set_aspect('equal', adjustable='box')

plt.xlim(-1.1, 1.1)
plt.ylim(-1.1, 1.1)
plt.xlabel("x")
plt.ylabel("y")

plt.suptitle(f"Estimate of Pi: {estimatePi(samples)}", fontsize = 18)
plt.title("Graphical Representation Is Independent To Estimate Calculation",
          fontsize = 10)
plt.show()

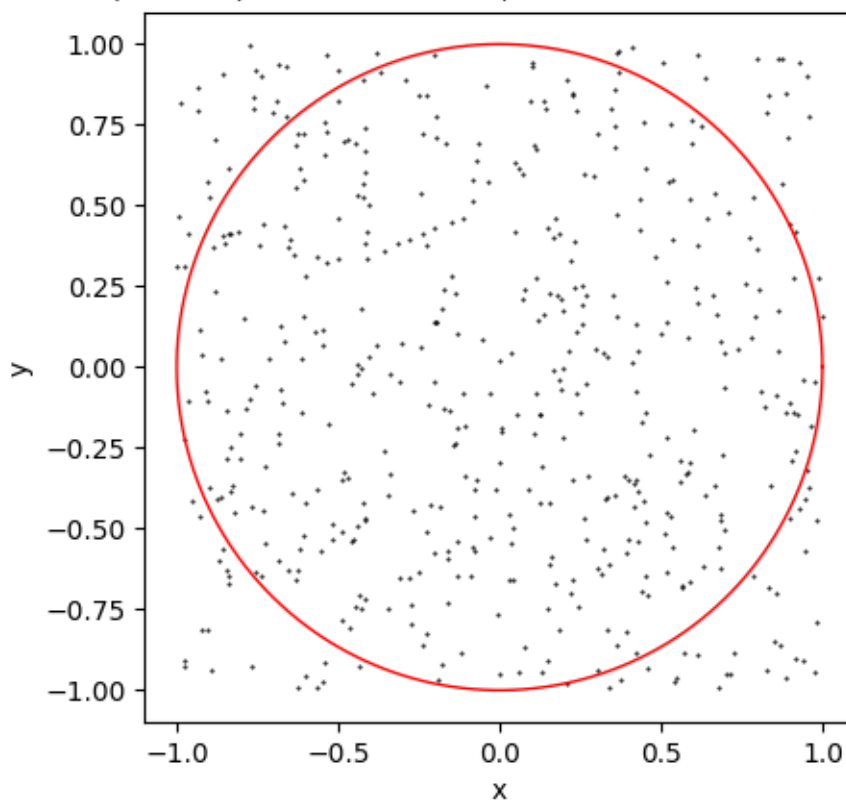
if __name__ == "__main__":
    main()
```

B Approximation of Pi in Python, Code Outputs

```
Estimate with 1 Sample:  
4.0  
  
Estimate with 10 Sample:  
3.2  
  
Estimate with 50 Sample:  
2.96  
  
Estimate with 100 Sample:  
3.04  
  
Estimate with 1000 Sample:  
3.104  
  
Estimate with 10,000 Sample:  
3.1376  
  
Estimate with 1,000,000 Sample:  
3.143152
```

Estimate of Pi: 3.152

Graphical Representation Is Independent To Estimate Calculation



C Lennard-Jones Potential, Simple Plot in Python

LennardJonesPotential/src/LennardJonesPotential.py

```
#!/ Python3
"""
Lennard-Jones potential plotted in Matplotlib
"""
import numpy as np
import matplotlib.pyplot as plt
from typing import final

def main():
    r = np.linspace(0, 3.0, 1000)
    SIGMA: final = 1.0
    EPSILON: final = 1.0
    V = 4 * EPSILON * ((SIGMA/r)**12 - (SIGMA/r)**6)

    plt.plot(r, V)

    plt.ylim(-1.2, 2)
    plt.xlim(0.9, 3)
    plt.grid(True)
    plt.ylabel("Potential Energy (V)")
    plt.xlabel("Particle Distance (r)")

    plt.show()

if __name__ == "__main__":
    main()
```

D Jensen's Inequality

4.9 Theorem (Jensen's inequality). *If g is convex, then*

$$\mathbb{E}g(X) \geq g(\mathbb{E}X). \quad (4.6)$$

If g is concave, then

$$\mathbb{E}g(X) \leq g(\mathbb{E}X). \quad (4.7)$$

PROOF. Let $L(x) = a + bx$ be a line, tangent to $g(x)$ at the point $\mathbb{E}(X)$. Since g is convex, it lies above the line $L(x)$. So,

$$\mathbb{E}g(X) \geq \mathbb{E}L(X) = \mathbb{E}(a + bX) = a + b\mathbb{E}(X) = L(\mathbb{E}(X)) = g(\mathbb{E}X). \quad \blacksquare$$

From Jensen's inequality we see that $\mathbb{E}(X^2) \geq (\mathbb{E}X)^2$ and if X is positive, then $\mathbb{E}(1/X) \geq 1/\mathbb{E}(X)$. Since $\log$ is concave, $\mathbb{E}(\log X) \leq \log \mathbb{E}(X)$.

via (Wasserman 2004)

E Monte Carlo C++ Code

ParticleSimulation/src/main.cpp

```
#include <iostream>
#include <vector>
#include <cmath>
#include <cstdlib>
#include <string>
#include <random>
#include <fstream>

// To simulate position of particles
struct Position {
    double x, y;
};

const std::string csvTracking_FilePath = "C:\\Users\\rajam\\OneDrive -
University of Reading\\THIRD YEAR\\PPR\\Numerical-simulation-methods-and-
their-applications\\ParticleSimulation\\data\\energy_data.csv";
const std::string rdfTracking_FilePath = "C:\\Users\\rajam\\OneDrive -
University of Reading\\THIRD YEAR\\PPR\\Numerical-simulation-methods-and-
their-applications\\ParticleSimulation\\data\\rdf_histogram.csv";

// Random Number Generation (Mersenne Twister)
// Using a seed of "32005096" for reproducibility

// std::random_device rd; // Random seed used during programming and testing
std::mt19937 gen(32005096);
std::uniform_real_distribution<double> uniform_dist(0.0, 1.0);

int initialisation(int NumParticles, double Density, std::vector<Position>&
particles, double& box_length) {

    // Calculation of the simulation box length
    // Density ( $\rho$ ) =  $N / \text{Area}$ . Therefore,  $\text{Area} = N / \text{Density}$ .
    // The length of one side of a square is the square root of its area.
    box_length = std::sqrt(NumParticles / Density);

    // Determining lattice size and spacing
    int n_cells = std::ceil(std::sqrt(NumParticles));
    double spacing = box_length / n_cells;

    // Looping through the grid and place particles
    int p = 0; // Particle counter

    for (int x = 0; x < n_cells; ++x) {
        for (int y = 0; y < n_cells; ++y) {

            // Only place a particle if NumParticles not yet reached
            if (p < NumParticles) {
                Position new_pos;
                new_pos.x = (x + 0.5) * spacing;
                new_pos.y = (y + 0.5) * spacing;

                particles.push_back(new_pos);
                p++;
            }
        }
    }
}
```

```

    }
}

std::cout << "Initialization complete. Placed " << particles.size() << "
particles.\n";
return 0;
}

double calc_energy(int idx, const std::vector<Position>& particles, double
box_length) {
    // Calculation of a Particle's Energy
    double energy = 0.0;
    double rc2 = 6.25; // Cutoff radius squared (2.5 * 2.5)

    for (int i = 0; i < particles.size(); ++i) {
        if (i == idx) continue; // prevent self-interaction of a particle

        // Calculate 2D distance components
        double dx = particles[idx].x - particles[i].x;
        double dy = particles[idx].y - particles[i].y;

        // Periodic Boundary Conditions (Minimum Image Convention)
        dx -= box_length * std::round(dx / box_length);
        dy -= box_length * std::round(dy / box_length);

        double r2 = dx*dx + dy*dy;

        // Lennard-Jones Potential Calculation (only if within cutoff)
        if (r2 < rc2) {
            double r6_inv = 1.0 / (r2 * r2 * r2);
            energy += 4.0 * (r6_inv * r6_inv - r6_inv);
        }
    }
    return energy;
}

// Test initial energy
double calc_total_energy(const std::vector<Position>& particles, double
box_length) {
    double total_energy = 0.0;
    for (int i = 0; i < particles.size(); ++i) {
        total_energy += calc_energy(i, particles, box_length);
    }
    // Divide by 2 to account for pairs
    return total_energy / 2.0;
}

void mcmove(std::vector<Position>& particles, double box_length, double Temp,
double max_disp, int& accepted_moves) {

    // Picking a random particle uniformly
    std::uniform_int_distribution<int> particle_dist(0, particles.size() - 1);
    int idx = particle_dist(gen);

    // Calculation of particle energy in current state
    double e_old = calc_energy(idx, particles, box_length);

```

```

    // Saving old position in case of rejection
    Position old_pos = particles[idx];

    // Random displacement proposition
    particles[idx].x += (uniform_dist(gen) - 0.5) * max_disp;
    particles[idx].y += (uniform_dist(gen) - 0.5) * max_disp;

    // PBC
    particles[idx].x -= box_length * std::floor(particles[idx].x / box_length);
    particles[idx].y -= box_length * std::floor(particles[idx].y / box_length);

    // Energy calculation in proposed new state
    double e_new = calc_energy(idx, particles, box_length);
    double delta_e = e_new - e_old;

    // Metropolis Acceptance Criteria
    // If delta_e negative, condition is true, accepted
    // If delta_e positive, Boltzmann probability checked
    if (delta_e < 0.0 || uniform_dist(gen) < std::exp(-delta_e / Temp)) {
        accepted_moves++; // Move accepted, particle stays
    } else {
        particles[idx] = old_pos; // Move rejected, og position restored
    }
}

void simulate_withTracking(int Cycles, int NumParticles, std::vector<Position>&
particles, double box_length, double Temp, double max_disp, int&
accepted_moves) {

    std::ofstream outfile(csvTracking_FilePath);
    outfile << "Cycle,TotalEnergy\n";

    // Setup for RDF Histogram variables
    int num_bins = 100;
    double max_r = box_length / 2.0; // Max radius to measure is half the box
    double bin_width = max_r / num_bins;
    std::vector<double> rdf_histogram(num_bins, 0.0);
    int rdf_samples = 0;

    std::cout << "Running " << Cycles << " MC cycles and writing data files...\n";

    for (int c = 0; c < Cycles; ++c) {
        for (int i = 0; i < NumParticles; ++i) {
            mcmmove(particles, box_length, Temp, max_disp, accepted_moves);
        }

        // Data Sampling (Every 10 cycles)
        if (c % 10 == 0) {
            double current_energy = calc_total_energy(particles, box_length);
            outfile << c << "," << current_energy << "\n";

            // RDF Sampling
            rdf_samples++;
            for (int i = 0; i < NumParticles - 1; ++i) {
                for (int j = i + 1; j < NumParticles; ++j) {

                    double dx = particles[i].x - particles[j].x;

```

```

        double dy = particles[i].y - particles[j].y;

        // PBC for distance measurement
        dx -= box_length * std::round(dx / box_length);
        dy -= box_length * std::round(dy / box_length);

        double r = std::sqrt(dx*dx + dy*dy);

        if (r < max_r) {
            int bin = static_cast<int>(r / bin_width);
            rdf_histogram[bin] += 2.0; // +2 doubled pairs
        }
    }
}

outfile.close();

// Write to csv
std::ofstream rdf_file(rdfTracking_FilePath);
rdf_file << "Distance,Count\n";
for (int b = 0; b < num_bins; ++b) {
    double r_center = (b + 0.5) * bin_width;
    double average_count = rdf_histogram[b] / rdf_samples;
    rdf_file << r_center << "," << average_count << "\n";
}
rdf_file.close();

std::cout << "Data collection complete. Files closed successfully.\n";
}

int main() {
    // Variables
    double Temp = 1.0;
    int NumParticles = 150;
    double Density = 0.8;
    int Cycles = 5000;
    double max_disp = 0.25; // Maximum allowed distance a particle can jump in
                           // one step

    std::vector<Position> particles;
    double box_length = 0.0;

    // Initialisation
    initialisation(NumParticles, Density, particles, box_length);
    std::cout << "Initial System Energy: " << calc_total_energy(particles,
        box_length) << "\n\n";

    // Simulation running
    int accepted_moves = 0;
    int total_moves_attempted = NumParticles * Cycles;

    simulate_withTracking(Cycles, NumParticles, particles, box_length, Temp,
        max_disp, accepted_moves);

    // Final Output
    std::cout << "\nFinal System Energy: " << calc_total_energy(particles,

```

```
        box_length) << "\n";
    std::cout << "Acceptance Ratio: " << (double)accepted_moves /
        total_moves_attempted * 100.0 << "%\n";

    return 0;
}
```